

Tecnologia espacial conectada a problemas reais da sociedade — agronegócio, sustentabilidade, comunicação e mobilidade impulsionados por dados orbitais e IA generativa.

Peso: 0 a 10

Esforço: ~5 horas

Individual · Dupla · Trio

4 trilhas temáticas

Ollama Cloud API

Tema GS — Indústria espacial · impacto terrestre · novos modelos de negócio.

Trilhas — 🌾 AgroSat · 🌳 EnviroSat · 📡 ConnectSat · 🚗 MobilitySat.

Stack — Python local · Ollama Cloud (gpt-oss:120b) · CLI estilo Claude Code.

Entregável — Repositório GitHub público + vídeo no YouTube + arquivo .txt no portal.

SUMÁRIO

- 01. Apresentação do desafio
- 02. Contexto: indústria espacial e IA generativa
- 03. O que vocês vão construir — Mission Control AI
- 04. Stack técnico obrigatório: Ollama Cloud local
- 05. Interface CLI estilo Claude Code
- 06. Estrutura oficial do projeto
- 07. Requisitos funcionais e técnicos
- 08. Formação de equipes
- 09. O que deve ser entregue
- 10. Critérios de avaliação (rubrica)
- 11. Checklist final antes de submeter
- 12. Cronograma e penalidades
- 13. Dicas e antipatterns
- 14. Contato e suporte

01 • APRESENTAÇÃO

Global Solution 2026.1 — Mission Control AI

Disciplina: Prompt Engineering and Artificial Intelligence · Curso: Ciência da Computação · Período: 1º Semestre de 2026

· Peso: 0 a 10

A **Global Solution 2026.1** desafia vocês a desenvolver soluções tecnológicas inspiradas nos problemas reais da **indústria espacial moderna**: exploração planetária, comunicação interplanetária, automação autônoma e segurança aeroespacial. O **Mission Control AI** é o nosso recorte: um sistema inteligente que recebe dados simulados de telemetria de uma missão espacial e usa um modelo de linguagem para interpretar, alertar e responder em linguagem natural.

Diferente de um exercício acadêmico abstrato, este desafio coloca vocês frente ao tipo de problema que define carreira em IA aplicada: **integrar lógica de domínio + IA generativa + interface mínima usável**, tudo em **até 5 horas de trabalho efetivo**, sem dependência de API paga.

POR QUE ESSE DESAFIO IMPORTA

Você não vai construir o próximo SpaceX em 5 horas — e essa não é a ideia. A ideia é exercitar, em pequena escala, exatamente o ciclo que define um engenheiro de IA: **entender o problema, escolher o prompt certo, integrar o LLM ao código, expor os dados de forma útil e validar que funciona**. Esse ciclo é o mesmo seja para missão espacial, e-commerce, ou saúde.

O projeto é avaliado em escala de **0 a 10**, e pode ser entregue em **modalidade individual, dupla ou trio**. Toda a stack técnica é gratuita: usaremos a **Ollama Cloud API** executada via script Python local (idêntico ao usado no Checkpoint 02 e Checkpoint 03), sem precisar instalar modelos no computador nem rodar Colab.

02 • CONTEXTO

Indústria Espacial e Impacto Terrestre

Conectando tecnologia orbital a problemas reais da sociedade

A exploração espacial está impulsionando uma **nova corrida tecnológica** capaz de transformar setores como **sustentabilidade, mobilidade, agronegócio, comunicação** e inteligência artificial. O que muitos ainda enxergam como ficção científica é, na verdade, infraestrutura crítica do nosso dia a dia: **cada vez que você usa Google Maps, recebe previsão de safra, ou assiste TV via satélite, está consumindo serviços de uma constelação orbital**.

Neste primeiro semestre de 2026, o tema norteador da Global Solution é a **Indústria Espacial** — e o desafio de vocês é desenvolver soluções inovadoras que **conectem tecnologia espacial e problemas reais da sociedade**, criando impacto positivo por meio de dados, automação, IA e novos modelos de negócio.

O ÂNGULO QUE ESTA GS PRIVILEGIA

Não basta construir um sistema que monitora satélites em órbita "porque é legal". O que diferencia projetos bons de projetos memoráveis é a clareza com que vocês articulam **quem se beneficia na Terra** quando esse satélite opera bem — e **quem sofre** quando ele falha. Sustentabilidade, agronegócio, mobilidade, conectividade: estas são as **razões de existir** da maior parte do que está em órbita hoje.

2.1 — O salto da IA generativa no espaço

Missões espaciais geram **volumes massivos de dados de telemetria** — temperatura de módulos, nível de energia, qualidade de sinal, integridade de payloads — que precisam ser interpretados em tempo real. É nesse gap entre **dado bruto orbital** e **decisão humana terrestre** que a IA generativa entra. Um LLM bem-prompted é capaz de:

INTERPRETAR TELEMETRIA

Transformar valores numéricos do satélite em diagnósticos compreensíveis para operadores e gestores não-especialistas.

DETECTAR ANOMALIAS

Cruzar múltiplos parâmetros simultaneamente e identificar padrões de risco que isoladamente passariam despercebidos.

TRADUZIR PARA IMPACTO

Explicar em linguagem natural o que cada anomalia significa para o cliente terrestre — produtor rural, gestor logístico, brigada ambiental.

Esse não é cenário futurista. A **NASA já usa LLMs internos** para análise de logs de missão e assistência a engenheiros do JPL. A **ESA** investiga IA para autonomia em sondas. O **setor de New Space brasileiro** (AEB, Visiona, Akaer, Aeropath e startups privadas) cresce em ritmo acelerado, e a demanda por engenheiros que sabem conectar dados orbitais a problemas terrestres é uma das maiores do mercado.

2.2 — Setores brasileiros onde IA + espaço já transformam realidades

AGRONEGÓCIO

Satélites de sensoriamento alimentam plataformas como Climate FieldView, Strider e Embrapa Monitora — usadas por milhares de produtores brasileiros para gestão de safras, irrigação e seguro rural baseado em índice.

SUSTENTABILIDADE E CLIMA

INPE opera DETER e PRODES com base em CBERS e Amazônia-1 — sistemas oficiais brasileiros de detecção de desmatamento e incêndios que fundamentam ações do IBAMA e brigadas estaduais.

COMUNICAÇÃO E INCLUSÃO

Constelações LEO (Starlink, OneWeb, futuras nacionais) estão levando internet a escolas rurais, postos de saúde isolados e comunidades ribeirinhas — viabilizando telemedicina e educação a distância.

MOBILIDADE E LOGÍSTICA

GNSS de alta precisão habilita agricultura de precisão (drones, plantadeiras autônomas), otimização de frotas logísticas e bases para veículos autônomos.

SUA TAREFA NESTA GS

Construir um **sistema de monitoramento de missão espacial** que: **(1)** simula a operação de um satélite real, **(2)** usa IA generativa para análise de telemetria e **(3)** articula explicitamente **como cada decisão técnica em órbita gera impacto terrestre** no setor escolhido pelo grupo. Não é fazer o sistema real de uma operadora — é demonstrar domínio do ciclo prompt → modelo → resposta contextualizada com clareza de propósito.

Mission Control AI — escolha sua trilha

4 trilhas temáticas, 1 escolha obrigatória, mesmo escopo de trabalho

Vocês vão construir um **sistema de monitoramento operacional** que recebe dados simulados de telemetria de um satélite real, detecta anomalias via lógica Python, e analisa o estado da missão em linguagem natural usando IA generativa. **Cada grupo escolhe UMA das 4 trilhas temáticas abaixo** — todas com o mesmo escopo de trabalho (~5 horas), a mesma rubrica e a mesma estrutura técnica de base.

A diferença entre as trilhas está em **(a) o tipo de satélite simulado, (b) os parâmetros de telemetria monitorados, (c) a persona terrestre atendida e (d) o setor da sociedade impactado**. Escolham a trilha que ressoa mais com os interesses do grupo:

3.1 — As 4 trilhas disponíveis

Trilha 1 — AgroSat (Sensoriamento Agrícola)

Satélite simulado: Satélite de sensoriamento multiespectral em órbita baixa, similar a CBERS-4A ou Planet Labs.

Parâmetros monitorados: saúde do sensor multiespectral (NDVI), temperatura do payload óptico, capacidade de armazenamento, janela de downlink, estabilidade de atitude.

Setor de impacto: Agronegócio brasileiro — gestão de safras, irrigação, seguro rural baseado em índice.

Personas atendidas: Engenheiro de operações do satélite, produtor rural consumidor do dado, analista de seguro agrícola.

Trilha 2 — EnviroSat (Observação Ambiental)

Satélite simulado: Satélite de observação ambiental com sensor térmico e óptico, similar a Amazônia-1 ou Landsat.

Parâmetros monitorados: sensor térmico (detecção de focos), sensor óptico RGB+NIR, buffer de imagens não transmitidas, precisão de geolocalização, energia disponível.

Setor de impacto: Sustentabilidade e clima — combate ao desmatamento, resposta rápida a incêndios, monitoramento de áreas protegidas.

Personas atendidas: Operador de centro de controle ambiental (INPE/órgão estadual), coordenador de brigada de combate a incêndio, analista de compliance ambiental.

Trilha 3 — ConnectSat (Conectividade Rural)

Satélite simulado: Satélite de telecomunicações em LEO, estilo Starlink, OneWeb ou futuras constelações nacionais.

Parâmetros monitorados: latência uplink, throughput do feixe, saúde da antena phased-array, beam steering, carga térmica do transponder.

Setor de impacto: Comunicação e inclusão digital — internet para escolas rurais, telemedicina, pequenos negócios sem fibra.

Personas atendidas: NOC engineer da operadora, coordenador de programa de inclusão digital, cliente final em comunidade rural.

Trilha 4 — MobilitySat (GNSS e Mobilidade)

Satélite simulado: Satélite GNSS de navegação, estilo GPS, Galileo ou GLONASS.

Parâmetros monitorados: drift do oscilador atômico, sincronização com a constelação, integridade do sinal L1/L5, precisão da efeméride, margem de potência.

Setor de impacto: Mobilidade e logística — frotas otimizadas, agricultura de precisão, base para veículos autônomos.

Personas atendidas: Engenheiro de segmento espacial, gestor de frota logística, operador de agricultura de precisão.

NÃO HÁ TRILHA MAIS FÁCIL OU MAIS DIFÍCIL

Todas as 4 trilhas têm o mesmo escopo de trabalho, a mesma rubrica e a mesma estrutura técnica de base. Escolham pelo **interesse do grupo**, não por percepção de dificuldade. A estrutura de pastas e o código-base de cada arquivo estão integralmente documentados neste documento (seção 06) — vocês criam o projeto seguindo essa estrutura e adaptam os parâmetros, a persona e o storytelling à trilha escolhida.

3.2 — O que o sistema deve fazer (todas as trilhas)

Independente da trilha escolhida, o sistema deve aplicar conceitos de **programação, algoritmos, pensamento computacional** e **IA generativa** para construir uma plataforma funcional de análise operacional. Ele deve contemplar:

- ☐ **Geração ou leitura** de dados simulados da telemetria do satélite (parâmetros específicos da trilha)
- ☐ **Geração automática de alertas** diante de parâmetros críticos
- ☐ **Estruturas lógicas de decisão** em código Python (não apenas no prompt)
- ☐ **Respostas automatizadas** diante de situações críticas simuladas
- ☐ **Uso de IA generativa** (Ollama Cloud) para análise contextualizada em linguagem natural
- ☐ **Conexão com o impacto terrestre** — cada alerta deve traduzir o que aquela anomalia significa para o

- ❑ **Visualização clara** via interface CLI estilo Claude Code (estrutura documentada na seção 06)

0 DIFERENCIAL 2026.1: CONECTAR COM A TERRA

Em 2025 essa GS pediria só "monitora telemetria e dispara alerta". Em 2026, o que diferencia projeto bom de projeto memorável é a clareza com que o sistema **articula o impacto terrestre** de cada alerta. O `system_prompt.md` da IA é onde vocês instruem o modelo a sempre amarrar análise técnica + consequência social, e a CLI deve ter um comando ou modo de visualização que dê palco para essa narrativa.

04 • STACK TÉCNICO

Ollama Cloud via Python local

A mesma stack usada no Checkpoint 02 e Checkpoint 03 — sem instalações pesadas

Esta é a única **stack obrigatória** do desafio. Não é necessário rodar modelos locais nem usar Google Colab. Vocês vão usar a **Ollama Cloud API** diretamente do seu computador, em qualquer IDE (VS Code, PyCharm, Cursor, ou até notebook .ipynb local), seguindo exatamente o mesmo padrão dos checkpoints anteriores.

POR QUE ESSA STACK

Ollama Cloud oferece o modelo **gpt-oss:120b** via API com plano gratuito generoso, sem necessidade de cartão de crédito. O acesso é via biblioteca `ollama` do Python — mesma interface familiar dos checkpoints. Nenhum modelo precisa rodar no PC do aluno, o que elimina os problemas de hardware insuficiente que apareceram em semestres anteriores.

4.1 — Setup inicial (passo a passo)

1 Criar conta gratuita no Ollama Cloud

Acessem <https://ollama.com>, criem uma conta gratuita e gerem uma **API Key** no painel de configurações. Guardem essa chave — ela será usada no arquivo `.env`.

2 Estrutura do projeto — siga o modelo da seção 06

A estrutura de pastas que vocês vão construir é a do **modelo oficial**, detalhada na [seção 06 — Estrutura do Projeto](#). Lá vocês encontram a árvore de pastas completa, com `src/telemetry.py`,

`src/alertas.py`, `src/engine.py`, `prompts/system_prompt.md` e os scripts auxiliares (`main.py`, `src/ui.py`, `banner_ascii.py`) — todos com código de partida documentado.

O que importa nesta seção 4 é apenas o **setup das credenciais** (passos **1**, **3** e **4**) — a organização do projeto vocês montam manualmente em um repositório novo seguindo a seção 06.

UMA ÚNICA ESTRUTURA OFICIAL

Não criem um repositório do zero com `mission_data.py` e `ai_analyzer.py` na raiz. **Sigam a estrutura de pastas e os arquivos da seção 06** (`src/`, `prompts/`, `data/`, `assets/`) — a correção é feita esperando essa organização.

3 Arquivo .env (NUNCA versionar)

.ENV

```
# Arquivo: .env (raiz do projeto)
# IMPORTANTE: adicione .env ao .gitignore ANTES de commitar!
OLLAMA_API_KEY=sua_chave_aqui_sem_aspas
```

ATENÇÃO — SEGURANÇA DA API KEY

Se a chave for commitada acidentalmente no GitHub, ela **continua no histórico mesmo após remoção**. Em caso de exposição, gerem nova chave imediatamente no painel da Ollama.

Penalidade automática: **-2 pontos** se a chave for encontrada no histórico do repositório.

4 requirements.txt

REQUIREMENTS.TXT

```
# Dependências mínimas — fixem as versões para reprodutibilidade
ollama==0.4.7
python-dotenv==1.0.1
# Opcional, dependendo do que vocês implementarem:
# rich==13.9.4          (terminal bonito)
```

```
# pandas=2.2.3      (manipulação de dados)
# matplotlib=3.9.2  (gráficos)
```

4.2 — Script padrão de integração com Ollama Cloud

Este é o **mesmo padrão usado no Checkpoint 02 e Checkpoint 03**. Vocês já viram esse código — basta adaptar ao contexto da missão espacial:

AI_ANALYZER.PY — CLIENTE OLLAMA CLOUD

```
import os
from ollama import Client
from dotenv import load_dotenv
import time
from datetime import datetime

load_dotenv()

# Configuração Ollama Cloud (mesma do Checkpoint 02 e 03)
client = Client(
    host="https://ollama.com",
    headers={'Authorization': 'Bearer ' + os.environ.get('OLLAMA_API_KEY')}
)
api = os.environ.get('OLLAMA_API_KEY')

print("API KEY carregada: ", "OK" if api else "FALTANDO")

def llm(prompt, max_tokens=800, temperature=0.3):
    """Envia prompt ao gpt-oss:120b via Ollama Cloud e retorna texto."""
    try:
        return client.chat(
            model="gpt-oss:120b",
            messages=[{"role": "user", "content": prompt}],
            options={"num_predict": max_tokens, "temperature": temperature},
            stream=False
        )['message']['content'].strip()
    except Exception as e:
        return f"⚠ Erro ao consultar IA: {e}"
```

A função `llm(prompt)` é o seu **ponto único de contato** com a IA. Vocês podem chamá-la de qualquer lugar do código passando o prompt construído dinamicamente com os dados da missão.

Copiem essa função `llm()` para o arquivo `src/engine.py` exatamente como mostrada acima. Ela é o **único ponto de contato** do projeto com o modelo — toda chamada à IA passa por ela. Não reescrevam essa função; chamem-na de dentro do `MissionEngine.analyze()` que vocês vão implementar (veja seção 06).

Chatbot via terminal estilo Claude Code

A interface já está pronta — vocês só implementam o motor de análise

A interface do chatbot deve ser via **terminal**, com banner em **ASCII art**, painéis renderizados e prompt editável — inspirada nas CLIs modernas como **Claude Code**, **OpenAI Codex CLI** e **Antigravity CLI**. Vocês **não precisam projetar essa interface do zero** — a [seção 06](#) documenta toda a estrutura do `src/ui.py` com o código de base. Vocês copiam esse código, adaptam ao tom da trilha escolhida, e concentram o esforço maior na lógica de telemetria, alertas e prompts.

PRINCÍPIO DO PROJETO

A camada de **apresentação (CLI)** está resolvida. O que vale nota é o que vocês fazem com a **camada de domínio**: como geram a telemetria, que regras de decisão escrevem em Python, e quão bom é o system prompt que vocês entregam à IA. A interface é só o palco — vocês escrevem a peça.

5.1 — Bibliotecas usadas na interface

Todas as bibliotecas abaixo devem entrar no `requirements.txt` do projeto. São as **opções mais maduras e atualizadas em 2026** para construir TUIs (Text User Interfaces) em Python:

STACK DA INTERFACE (JÁ CONFIGURADA)

BIBLIOTECA	VERSÃO	FUNÇÃO NO PROJETO
Rich	15.0.0	Renderização de painéis com bordas, tabelas, markdown, syntax highlighting e spinners no terminal. É a biblioteca por trás dos painéis bonitos que vocês veem no Claude Code.
prompt-toolkit	3.0.52	Input editável com histórico (setas ↑↓), auto-completar, multilinha e estilização do cursor. É o que dá o "feel" de CLI moderna.

PyFiglet

1.0.4

Gera o banner em ASCII art da frase "Mission Control AI". Inclui mais de 570 fontes prontas.

python-dotenv

1.2.2

Carrega a chave Ollama do arquivo `.env` sem deixar credenciais hardcoded.

ollama

0.6.2

Cliente oficial Python para Ollama Cloud — modelo `gpt-oss:120b`.

Bibliotecas **opcionais** que vocês podem adicionar se quiserem incrementar (descomentem no `requirements.txt`):

TEXTUAL

TUI completa com layouts CSS-like, widgets reativos e animações. Use se quiser uma interface ainda mais sofisticada (estilo aplicativo). Mais complexo de aprender.

RICH-PYFIGLET

Wrapper do PyFiglet integrado ao Rich. Permite banners ASCII com gradientes de cor animados.

PANDAS + MATPLOTLIB

Para vocês que querem incrementar com séries temporais da telemetria e gráficos no terminal.

5.2 — Como o aluno vê a interface ao rodar `python main.py`

Ao executar o projeto pela primeira vez (mesmo sem ter implementado nada da lógica), o aluno verá **imediatamente**:

1. O banner ASCII colorido em ciano "MISSION CONTROL AI"
2. Um card com bordas listando os comandos e o modelo configurado
3. Um aviso amarelo: "Engine status: AGUARDANDO IMPLEMENTAÇÃO X"
4. Um prompt `>` pronto para receber input

Quando o aluno digitar qualquer pergunta (ex: "Como está a missão?"), o sistema responde com uma **mensagem amigável** avisando que a implementação está pendente, listando os arquivos que precisam ser completados. Conforme o aluno implementa `src/engine.py`, a mesma interface passa a retornar análises reais sem precisar mudar nada na camada de UI.

5.3 — Script auxiliar: `banner_ascii.py`

Como script auxiliar standalone, o `banner_ascii.py` gera a frase "Global Solution 2026.1 — Mission Control AI" em ASCII art. É útil para experimentar fontes e customizar o banner do seu projeto. Vocês criam esse script seguindo o código abaixo:

```
# Rodar o banner padrão
$ python banner_ascii.py

# Listar as 570+ fontes disponíveis no PyFiglet
$ python banner_ascii.py --fonts

# Testar uma fonte específica
$ python banner_ascii.py --font slant --text "Mission Control AI"

# Demonstrar 8 fontes diferentes lado a lado
$ python banner_ascii.py --demo
```

O conteúdo essencial do script (para vocês entenderem como o banner é gerado):

```
import pyfiglet
from rich.console import Console
from rich.align import Align
from rich.text import Text

console = Console()

# Gera as duas linhas do banner em ASCII art
linha1 = pyfiglet.figlet_format("Global Solution", font="ansi_shadow")
linha2 = pyfiglet.figlet_format("Mission Control AI", font="ansi_shadow")

# Pinta em ciano (estilo Claude Code) e centraliza
console.print(Align.center(Text(linha1, style="bold #A855F7")))
console.print(Align.center(Text(linha2, style="bold #06B6D4")))
console.print(Align.center(
    Text("— 2026.1 · Prompt Engineering and AI · FIAP —",
        style="italic #8484A0")
))
```

POR QUE ASCII ART IMPORTA AQUI

O banner ASCII é mais que decoração — ele **define a identidade da CLI** e é a primeira coisa que o avaliador vê no vídeo de demonstração. Um banner caprichado mostra atenção a detalhes e indica que o grupo se importou com o resultado final, não só com fazer "funcionar de qualquer jeito".

O modelo que vocês vão construir — pasta por pasta, arquivo por arquivo

6.1 — Árvore de pastas do projeto

Vocês criam um repositório novo no GitHub e montam a estrutura abaixo manualmente. Toda a documentação do código de cada arquivo está neste documento — copiem, adaptem ao contexto da trilha escolhida (AgroSat, EnviroSat, ConnectSat ou MobilitySat) e implementem em cima.

ESTRUTURA OFICIAL — SIGA EXATAMENTE ESTA ORGANIZAÇÃO

```
mission-control-ai/
|
├─ README.md           # Doc do grupo + persona + proposta de valor
├─ main.py             # Entrada do sistema (código na seção 6.2)
├─ banner_ascii.py     # Gerador de banner ASCII (código na seção 5.3)
├─ requirements.txt    # Dependências fixadas (código na seção 4.1)
├─ .env.example        # Template das variáveis (sem chave real)
├─ .env               # Chave Ollama (NUNCA commitar — fica no .gitignore)
├─ .gitignore         # Ignora .env, __pycache__, .venv
|
├─ src/
|   └─ __init__.py     # Arquivo vazio — marca src/ como pacote Python
|   └─ ui.py           # Chat CLI estilo Claude Code (código na seção 6.3)
|   └─ engine.py       # Motor de análise — vocês implementam (seção 6.4)
|   └─ telemetria.py   # Geração dos dados simulados — vocês implementam
|   └─ alertas.py      # Thresholds e regras de decisão — vocês implementam
|
├─ prompts/
|   └─ system_prompt.md # System prompt da IA — vocês escrevem
|
├─ data/
|   └─ cenarios.json   # OPCIONAL — cenários pré-definidos para teste
|
└─ assets/
    └─ screenshot_banner.png # Print do banner inicial do seu sistema
    └─ screenshot_análise.png # Print da IA analisando dados reais
```

IMPORTANTE: PROJETO PYTHON, NÃO NOTEBOOK

A entrega deve ser um **projeto Python estruturado em pastas e módulos (.py)**, como em qualquer aplicação real. **Não aceitamos entrega como notebook único Jupyter/Colab.** Notebooks podem ser

usados apenas para experimentação durante o desenvolvimento, mas o produto final precisa ser um projeto Python executável via `python main.py` seguindo a árvore acima.

6.2 — Arquivo `main.py` (entrada do sistema)

O `main.py` é o ponto de entrada. Ele apenas instancia o motor (`MissionEngine`) e passa o controle para a UI. Vocês copiam esse código quase como está — a lógica real fica em `src/engine.py`.

```
MAIN.PY

"""Mission Control AI - ponto de entrada do sistema."""
from src.ui import run_cli
from src.engine import MissionEngine

if __name__ == "__main__":
    engine = MissionEngine()
    run_cli(engine)
```

6.3 — Arquivo `src/ui.py` (interface CLI)

A UI já vem com o esqueleto pronto — vocês copiam e podem customizar cores, comandos extras e o tom dos painéis. A função `run_cli(engine)` recebe o motor, exibe o banner, gerencia o loop de input e despacha cada pergunta para `engine.analyze()`. Comandos suportados de base: `/help`, `/status`, `/about`, `/clear`, `/exit`.

```
SRC/UI.PY - ESQUELETO

"""Interface CLI estilo Claude Code - usa Rich + prompt-toolkit."""
from rich.console import Console
from rich.panel import Panel
from rich.text import Text
from prompt_toolkit import PromptSession
from prompt_toolkit.styles import Style
import pyfiglet
from datetime import datetime

console = Console()
session = PromptSession(style=Style.from_dict({"prompt": "#06B6D4 bold"}))

def show_banner():
    """Exibe banner ASCII colorido no início."""
    banner = pyfiglet.figlet_format("Mission Control", font="ansi_shadow")
```

```

console.print(Text(banner, style="bold #06B6D4"))
console.print(Panel.fit(
    "Sistema de monitoramento e análise por IA generativa.\n"
    "Use /help para ver os comandos · /exit para sair.\n"
    "Modelo: gpt-oss:120b via Ollama Cloud",
    title="◆ MISSION CONTROL", border_style="#06B6D4"
))

def show_response(text):
    """Renderiza resposta da IA em painel com timestamp."""
    now = datetime.now().strftime("%H:%M")
    console.print(Panel(text, title="◆ Mission Control",
                        subtitle=now, border_style="#06B6D4"))

def run_cli(engine):
    """Loop principal da CLI."""
    show_banner()
    if not engine.is_ready():
        console.print(" ⚠ Engine status: AGUARDANDO IMPLEMENTAÇÃO x\n",
                      style="yellow")

    while True:
        try:
            user_input = session.prompt("> ").strip()
        except (KeyboardInterrupt, EOFError):
            break

        if not user_input:
            continue

        if user_input == "/exit":
            break

        if user_input == "/help":
            console.print("Comandos: /help /status /about /clear /exit")
            continue

        if user_input == "/status":
            show_response(engine.status_snapshot())
            continue

        if user_input == "/clear":
            console.clear(); show_banner(); continue

        # Qualquer outra entrada vai para o motor de análise
        resposta = engine.analyze(user_input)
        show_response(resposta)

```

6.4 — Arquivo `src/engine.py` (motor de análise — foco do trabalho)

Este é o arquivo central do projeto. Ele combina **(a)** a função `llm()` que vocês já viram na seção 4.2 com **(b)** a classe `MissionEngine` que vocês vão implementar. O esqueleto começa com stubs que retornam mensagens amigáveis avisando o que falta — conforme vocês implementam, o sistema ganha vida sem precisar mudar nada na UI.

```
SRC/ENGINE.PY — ESQUELETO INICIAL

"""Motor de análise da Mission Control AI."""
import os
from ollama import Client
from dotenv import load_dotenv
from pathlib import Path

load_dotenv()

# Identificação da trilha — ALTEREM conforme a escolha do grupo
TRILHA = "envirosat" # "agrosat" | "envirosat" | "connectsat" | "mobilitysat"

client = Client(
    host="https://ollama.com",
    headers={'Authorization': 'Bearer ' + os.environ.get('OLLAMA_API_KEY', '')}
)

def llm(prompt, system=None, max_tokens=800, temperature=0.3):
    """Envia prompt ao gpt-oss:120b via Ollama Cloud."""
    messages = []
    if system:
        messages.append({"role": "system", "content": system})
    messages.append({"role": "user", "content": prompt})
    try:
        return client.chat(
            model="gpt-oss:120b", messages=messages,
            options={"num_predict": max_tokens, "temperature": temperature},
            stream=False
        )['message']['content'].strip()
    except Exception as e:
        return f"⚠ Erro ao consultar IA: {e}"

def load_system_prompt():
    """Lê o system prompt do arquivo prompts/system_prompt.md"""
    path = Path("prompts/system_prompt.md")
    if path.exists():
        return path.read_text(encoding="utf-8")
    return "Você é um assistente." # fallback genérico
```

```

class MissionEngine:
    """Motor de análise — vocês completam os métodos abaixo."""

    def __init__(self):
        self.trilha = TRILHA
        self.system_prompt = load_system_prompt()

    def is_ready(self):
        # Troquem para True quando analyze() estiver implementado
        return False

    def status_snapshot(self):
        """Retorna texto resumindo o estado atual da telemetria."""
        # TODO: chamar telemetria.coletar() e formatar legivelmente
        return "🔧 status_snapshot() ainda não implementado."

    def analyze(self, pergunta_usuario):
        """Analisa a pergunta com base na telemetria + alertas + IA."""
        # TODO (foco do trabalho):
        # 1. Coletar dados via src.telemetria.coletar()
        # 2. Avaliar alertas via src.alertas.avaliar(dados)
        # 3. Montar prompt com dados + alertas + pergunta
        # 4. Chamar llm(prompt, system=self.system_prompt)
        # 5. Retornar a resposta
        return (
            "🔧 Implementação pendente.\n\n"
            "Olá! A interface CLI está funcionando, mas a lógica\n"
            "de análise ainda não foi conectada. O grupo precisa:\n\n"
            " 1. Completar src/telemetria.py\n"
            " 2. Completar src/alertas.py\n"
            " 3. Escrever o system prompt em prompts/system_prompt.md\n"
            " 4. Sobrescrever analyze() em src/engine.py"
        )

```

6.5 — O que vocês implementam (foco do trabalho)

✓ JÁ DOCUMENTADO (COPIEM)

- **main.py** — código completo na seção 6.2
- **src/ui.py** — esqueleto na seção 6.3
- **banner_ascii.py** — código na seção 5.3
- **requirements.txt** — na seção 4.1
- **.env.example** e **.gitignore** — seção 4.1

⚙️ A IMPLEMENTAR (FOCO DO TRABALHO)

- **src/telemetria.py** — geração dos dados simulados
- **src/alertas.py** — thresholds e regras de decisão
- **prompts/system_prompt.md** — system prompt da IA
- Sobrescrever `MissionEngine.analyze()` em `src/engine.py`

- Função `llm()` e esqueleto da `MissionEngine` — seção 6.4

- Sobrescrever `MissionEngine.status_snapshot()`
- Trocar `is_ready()` para `True`
- Preencher **README.md** com info do grupo
- Capturar prints reais em **assets/**

6.6 – Comportamento esperado ao rodar `python main.py`

Logo após montar a estrutura e instalar as dependências, o sistema deve abrir a interface completa e **responder com mensagens amigáveis** apontando o que ainda falta implementar — exatamente o comportamento dos stubs mostrados na seção 6.4:

EXEMPLO DE INTERAÇÃO AO RODAR PELA PRIMEIRA VEZ

[banner ASCII em ciano da Mission Control AI]

⚠ Engine status: AGUARDANDO IMPLEMENTAÇÃO x

> Como está a missão?

```
> /help
```

WORKFLOW RECOMENDADO

1. Criem um repositório novo no GitHub do grupo. 2. Montem a árvore de pastas da seção 6.1. 3. Copiem o código dos arquivos já documentados (`main.py` , `src/ui.py` , `banner_ascii.py` , `requirements.txt` , `.env.example` , `.gitignore` , esqueleto do `src/engine.py`). 4. Configurem `.env` com sua chave Ollama. 5. Rodem `python main.py` e validem que a interface CLI abre. 6. Implementem `src/telemetria.py` com 3-4 parâmetros simulados. 7. Implementem `src/alertas.py` com 2-3 regras de threshold. 8. Escrevam o `prompts/system_prompt.md` (a parte mais importante). 9. Sobrescrevam `MissionEngine.analyze()` para amarrar tudo. 10. Testem cenários extremos. 11. Capturem prints. 12. Gravem o vídeo.

07 • REQUISITOS

Requisitos funcionais e técnicos

O que o projeto OBRIGATORIAMENTE precisa ter para receber nota

5.1 — Requisitos funcionais (obrigatórios)

🚀 Monitoramento de dados simulados

- Sistema deve **gerar ou ler** dados simulados de pelo menos **3 parâmetros** da missão (temperatura, energia, comunicação são o mínimo)
- Dados podem ser gerados aleatoriamente, lidos de um JSON/CSV, ou simulados em série temporal (ciclos)
- Não precisa ser realista cientificamente — precisa ser **plausível e coerente** com o cenário

⚠️ Lógica de alertas e decisão

- Pelo menos **uma lógica de threshold** implementada em código (ex: `if temperatura > 60: alerta_critico()`)
- Pelo menos **uma resposta automatizada** para situação crítica simulada (ex: ativar modo economia se energia < 20%)

- Lógica deve estar **no código Python**, não apenas no prompt da IA

Integração com IA generativa (Ollama Cloud)

- Modelo de linguagem **integrado e funcionando** via Ollama Cloud (gpt-oss:120b)
- **System prompt customizado** com contexto da missão espacial — não genérico
- Resposta da IA **exibida de forma legível** no output
- Pelo menos **1 chamada efetiva ao LLM** com dados reais da telemetria injetados no prompt

Visualização clara dos dados

- Saída organizada e **visualmente compreensível** (terminal estruturado é suficiente)
- Status atual da missão exibido com clareza (formatação ASCII, separadores, emojis OK)
- **Interfaces web/dashboards são OPCIONAIS** — não somam pontos extras automaticamente, mas podem entrar nos diferenciais

5.2 — Requisitos técnicos (obrigatórios)

LINGUAGEM

Python 3.10 ou superior. Comentários em **português brasileiro**.

MODELO DE IA

Ollama Cloud (`gpt-oss:120b`) via biblioteca `ollama` . Outros provedores gratuitos (Groq, Gemini) são aceitos mas **não recomendados** — fora do padrão da disciplina.

CREDENCIAIS

Sempre via `.env` + `python-dotenv` . Nunca hardcoded. `.env` deve estar no `.gitignore` .

AMBIENTE

Roda **localmente** em qualquer IDE. Notebook Jupyter local também aceito. Google Colab opcional.

VERSIONAMENTO

Git + GitHub público. Commits regulares ao longo do desenvolvimento — não tudo na noite da entrega.

REQUIREMENTS.TXT

Obrigatório, com versões fixadas. README deve indicar como instalar e executar.

5.3 — Diferenciais valorizados (não obrigatórios)

Elementos abaixo **não são obrigatórios** e nenhum vale ponto isoladamente — mas projetos que demonstram domínio de um ou mais desses elementos têm vantagem na avaliação subjetiva da rubrica:

 **FEW-SHOT PROMPTING**
Incluir exemplos de análise no prompt para guiar o modelo (técnica vista nas aulas de PE).

 **MÚLTIPLAS CHAMADAS LLM**
Encadear chamadas: uma para classificar severidade, outra para gerar plano de ação.

 **MEMÓRIA DE CONTEXTO**
Manter histórico dos últimos ciclos e passar para a IA, simulando consciência temporal.

 **SAÍDA ESTRUTURADA**
Forçar a IA a responder em JSON estruturado e parsear no Python para usar nos alertas.

 **MÚLTIPLAS PERSONAS**
Sistema que alterna entre persona operador / astronauta / engenheiro conforme o evento.

 **INTERFACE VISUAL**
Streamlit, Gradio, Rich (terminal colorido) ou matplotlib para visualização gráfica dos dados.

IMPORTANTE: FOCO NO QUE IMPORTA

Vocês têm **5 horas de trabalho efetivo**. Não tentem implementar tudo da lista de diferenciais — escolham **1 ou 2** que realmente agreguem ao projeto. Sistema simples e bem feito vale mais que sistema cheio de funcionalidades meia-boca.

08 • EQUIPES

Formação de equipes

Indivíduo, dupla ou trio — escolham o formato que se adapta ao perfil de vocês

A modalidade do trabalho é **flexível** e cada formato tem suas vantagens. Não há diferença na rubrica de avaliação entre as modalidades — o sistema entregue será avaliado com os mesmos critérios independente do tamanho do grupo.

MODALIDADES PERMITIDAS		
MODALIDADE	PERMITIDO?	QUANDO FAZ SENTIDO

Individual	✅ Sim	Quando você prefere trabalhar no seu ritmo e tem domínio confortável de Python
Dupla	✅ Sim	Equilíbrio ideal entre divisão de trabalho e baixa fricção de comunicação
Trio	✅ Sim	Permite especialização: um foca em dados, outro em prompts, outro em interface
Grupos maiores (4+)	❌ Não	Concentração de trabalho em poucos integrantes — evitar

Para **duplas e trios**, recomendo dividir responsabilidades de forma clara desde o início. Sugestão:

 DADOS & LÓGICA Geração de telemetria, thresholds, alertas, persistência de histórico.	 PROMPT ENGINEERING System prompt, iteração de respostas, ajuste de parâmetros do modelo.	 APRESENTAÇÃO Output no terminal, README, prints, gravação do vídeo demonstrativo.
--	--	---

TRABALHO COLABORATIVO, NÃO DUPLICADO

Vocês podem (e devem) discutir abordagens com colegas de outras equipes — isso é parte do aprendizado. Mas **arquivos idênticos entre equipes diferentes** serão considerados cópia. Cada grupo entrega seu próprio system prompt, sua própria lógica, sua própria implementação.

09 • ENTREGA

O que deve ser entregue

Um único arquivo `.txt` no portal — simples e auditável

A entrega final será feita **pelo portal da instituição**, em um único arquivo `.txt` contendo as informações abaixo. Esse padrão é o mesmo da Global Solution institucional e do Checkpoint 02/03:

```

CONTEÚDO DO ARQUIVO .TXT

# Arquivo: GS_2026_1_GrupoXX_TURMA.txt

Nome completo e RM de todos os integrantes:
- [Nome Completo] — RM: [número]
- [Nome Completo] — RM: [número]

```

- [Nome Completo] – RM: [número] *(se trio)*

Modalidade: [Individual | Dupla | Trio]

Link do repositório GitHub: <https://github.com/usuario/mission-control-ai>

Link do vídeo de demonstração: <https://www.youtube.com/watch?v=...>

9.1 — O repositório GitHub deve conter

- ☐ **README.md** completo na raiz (estrutura detalhada na seção 9.2)
- ☐ **Código Python funcional** rodável após `pip install -r requirements.txt`
- ☐ **requirements.txt** com versões fixadas
- ☐ **.env.example** mostrando quais variáveis são necessárias (sem a chave real)
- ☐ **.gitignore** configurado para ignorar `.env` e `__pycache__`
- ☐ **Pasta assets/** com no mínimo 2 prints reais do sistema funcionando
- ☐ Repositório **público** — sem isso, a entrega é considerada não realizada

9.2 — Estrutura mínima do README

🚀 Mission Control AI – [Nome do Projeto]

Integrantes

- Nome Completo – RM: XXXXXX – Turma: XCCXX
- Nome Completo – RM: XXXXXX – Turma: XCCXX

O que o projeto faz

[2-3 frases descrevendo o sistema e como a IA está integrada]

Persona atendida

[Operador / Astronauta / Engenheiro / Outra – justifique em 1-2 frases]

Tecnologias utilizadas

- Python 3.10+
- Ollama Cloud API (modelo gpt-oss:120b)
- Bibliotecas: ollama, python-dotenv, [outras]

Como executar

1. Clone o repositório
2. Crie ambiente virtual: ``python -m venv .venv && source .venv/bin/activate``
3. Instale dependências: ``pip install -r requirements.txt``
4. Crie arquivo ``.env`` na raiz com:
``OLLAMA_API_KEY=sua_chave_aqui``
5. Execute: ``python main.py``

Demonstração

![Status normal da missão](assets/screenshot_normal.png)
![Alerta crítico com análise da IA](assets/screenshot_alerta.png)

System Prompt

[Cole aqui o system prompt usado, ou linke para prompts/system_prompt.md]

Cenários de teste demonstrados

1. Operação normal — todos os parâmetros dentro do range
 2. Temperatura crítica — alerta + análise IA
 3. Perda de comunicação — resposta automatizada
- [adicione outros cenários que seu sistema cobre]

Limitações conhecidas

[Liste honestamente o que o sistema NÃO faz — pontos por honestidade técnica]

Vídeo de demonstração

 [Assistir no YouTube](https://www.youtube.com/watch?v=...)

9.3 — Vídeo de demonstração

O vídeo de demonstração deve ser **obrigatoriamente publicado no YouTube** e o link deve aparecer em **dois lugares**: dentro do arquivo `.txt` enviado no portal **e também no `README.md` do repositório GitHub**. Os dois lugares são checados na correção; se faltar em um deles, perde pontos.

- ☐ Plataforma: **YouTube** (não aceitamos Drive, Vimeo, link direto, etc.)
- ☐ Privacidade: **"Não listado"** — não público, não privado
- ☐ Link presente no arquivo `.txt` enviado no portal
- ☐ Link presente **também no `README.md`** do repositório, com texto descritivo (ex: *"🎬 Assistir demonstração no YouTube"*)
- ☐ Duração: **máximo 3 minutos** (vídeos acima sofrem desconto)
- ☐ Apresentar os **integrantes do grupo** no início — pode ser por **rosto na câmera**, **voz narrando os nomes** ou **texto sobre frame** com nome e RM de cada integrante. As três formas são aceitas; ninguém é obrigado a

aparecer no vídeo.

- ☐ Demonstrar o **sistema rodando ao vivo** (não slides, não código parado)
- ☐ Mostrar a **IA gerando análise em tempo real** com base nos dados
- ☐ Demonstrar **pelo menos um cenário de alerta** ou situação crítica
- ☐ Áudio e imagem com **qualidade audível e legível**

TRECHO DO README.MD COM O LINK DO VÍDEO

```
## 📺 Vídeo de demonstração

🔗 [Assistir demonstração no YouTube](https://www.youtube.com/watch?v=SEU_ID_AQUI)

> Configurado como "Não listado" no YouTube.
```

PRIVACIDADE DO VÍDEO

Modo "não listado" significa que o vídeo **não aparece em buscas** mas é acessível por link. Vídeos **privados** (que exigem permissão), **removidos** antes da correção, ou hospedados **fora do YouTube** contam como ausentes — zero na frente do vídeo.

9.4 — Condições de entrega

As regras abaixo são **inegociáveis** e estão alinhadas à política institucional da FIAP. Leiam com atenção **antes** da data da entrega — depois não há como reverter:

POLÍTICA INSTITUCIONAL DE ENTREGA

- A integridade do arquivo é **responsabilidade do grupo**. Confirmam o conteúdo do `.txt` após o upload no portal.
- Arquivos **vazios, corrompidos** ou **repositórios privados sem acesso ao professor** não serão considerados.
- **Não serão aceitos envios pelo Teams**, e-mail, WhatsApp ou qualquer canal alternativo — apenas pelo portal institucional, dentro do prazo.
- **Entregas fora do prazo** só são aceitas com justificativa prévia aprovada pelo professor (atestado médico ou similar conforme normas FIAP).

Checklist de pré-submissão

- ☐ Repositório GitHub está **público** (testem em janela anônima)
- ☐ Vídeo do YouTube está como "**Não listado**" e o link funciona em janela anônima
- ☐ Arquivo `.txt` contém todos os campos: nomes, RMs, modalidade, link GitHub, link YouTube
- ☐ O mesmo link do YouTube está duplicado no `README.md` do repositório
- ☐ Upload feito no portal da instituição **dentro do prazo**

Dica final: faça o upload pelo menos **6 horas antes** do prazo. Problemas de internet, indisponibilidade do portal e bugs de upload acontecem — não confiem no último minuto.

10 • AVALIAÇÃO

Critérios de avaliação (rubrica)

Pontuação total: 0 a 10 — peso na nota da disciplina conforme calendário

A avaliação é dividida em **6 frentes**, cada uma com pontuação parcial somando **10 pontos** no total. Os critérios são objetivos sempre que possível e a rubrica abaixo é a referência usada na correção. Leiam **antes de iniciar o projeto**, não depois.

DISTRIBUIÇÃO DA NOTA (0 A 10)

#	FRENTE AVALIADA	PESO
1	Entrega correta — arquivo .txt com nomes/RMs, link GitHub e link do vídeo funcionando	1,0 pt
2	README completo — todas as seções, prints reais, instruções de execução	1,5 pts
3	Uso de IA via Ollama Cloud — modelo integrado, system prompt contextual, resposta coerente	3,0 pts
4	Funcionalidades da missão — dados simulados, alertas, lógica de decisão funcionando	2,5 pts
5	Vídeo de demonstração — mostra sistema funcionando, dentro de 3 minutos	1,0 pt
6	Proposta de valor / modelo de negócio — articula impacto terrestre e quem se beneficia	1,0 pt

TOTAL**10,0
pts**

10.1 — Como cada frente é avaliada

Frente 1 – Entrega correta (1,0 pt)

Arquivo `.txt` está correto, com todos os dados e links funcionando no dia da correção. Modalidade declarada corretamente.

SITUAÇÃO	PONTUAÇÃO
.txt completo, GitHub público, vídeo acessível	0,8–1,0
.txt presente mas com algum link quebrado	0,4–0,7
.txt incompleto ou faltando informações	0,1–0,3
.txt ausente	0,0

Frente 2 – README completo (1,5 pts)

Tem descrição do projeto, integrantes com RM, pelo menos 2 prints reais do sistema rodando, lista de tecnologias e instruções claras de execução. README vazio ou copiado = desconto direto.

SITUAÇÃO	PONTUAÇÃO
README completo, prints reais, instruções claras, persona justificada	1,3–1,5
README presente mas com pelo menos uma seção fraca (ex: 1 print só, ou instruções genéricas)	0,7–1,2
README mínimo — só descrição + integrantes	0,3–0,6
README ausente ou em branco	0,0

Frente 3 – Uso de IA via Ollama Cloud (3,0 pts)

O projeto deve usar o modelo de linguagem via Ollama Cloud integrado ao código, com prompt que faça **sentido para o contexto da missão espacial**. Outras APIs gratuitas são aceitas, mas a integração Ollama é o padrão.

SITUAÇÃO	PONTUAÇÃO
IA integrada, respondendo, system prompt rico com contexto da missão, dados injetados dinamicamente	2,5–3,0
IA integrada mas prompt genérico (sem contexto espacial profundo) ou sem injeção dinâmica de dados	1,5–2,4
Tentativa de integração com erros ou IA chamada mas sem usar os dados da missão	0,5–1,4
IA não integrada ou não funciona	0,0–0,4

Frente 4 – Funcionalidades da missão (2,5 pts)

O sistema exibe dados simulados, tem pelo menos uma lógica de alerta/decisão funcionando, e demonstra estruturas computacionais sólidas. Não precisa ser complexo — precisa **funcionar e ter substância**.

SITUAÇÃO	PONTUAÇÃO
3+ parâmetros monitorados, alertas funcionais, lógica de decisão clara, resposta automatizada para crise	2,1–2,5
Dados simulados funcionando mas com lógica de alerta limitada (só 1 threshold simples)	1,3–2,0
Apenas exibição estática dos dados, sem lógica de decisão visível	0,4–1,2
Nada funcional	0,0–0,3

Frente 5 – Vídeo de demonstração (1,0 pt)

Mostra o grupo (nome ou rosto), executa o sistema ao vivo e demonstra pelo menos um cenário de alerta com a IA respondendo. Qualidade de câmera não é critério — clareza e completude sim.

SITUAÇÃO	PONTUAÇÃO
Vídeo até 3min, integrantes apresentados, sistema rodando, IA mostrada, alerta demonstrado	0,8–1,0
Vídeo presente mas pula algum critério (ex: não apresenta integrantes, ou não mostra alerta)	0,4–0,7
Vídeo muito curto (< 1min) ou de baixíssima qualidade que prejudica avaliação	0,1–0,3
Vídeo ausente ou inacessível na data da correção	0,0

Frente 6 – Proposta de valor / modelo de negócio (1,0 pt) – NOVIDADE 2026.1

O grupo articula explicitamente **quem se beneficia na Terra** quando a missão opera bem, e **quem paga** pela operação. A seção "📁 Proposta de valor / modelo de negócio" do README é o lugar dessa articulação. Não precisa ser plano de negócio completo — precisa ter **clareza de propósito**.

O grupo deve responder 4 perguntas no README:

1. Qual o **problema real terrestre** que esta missão resolve?
2. **Quem paga** pela solução? Setor público (governo, INPE, ANATEL)? Setor privado (operadora, cooperativa, frota)? Híbrido?
3. **Métrica de impacto:** se o satélite operar 100% saudável por 1 ano, o que muda concretamente no mundo? (Ex: X hectares monitorados, Y toneladas de CO₂ evitadas, Z escolas conectadas)
4. **Modelo de negócio:** SaaS? Dado-como-serviço? Concessão pública? Assinatura?

CALIBRAGEM DE ESFORÇO – FRENTE 6

Esta frente vale **1,0 pt**, mas exige reflexão diferente das frentes técnicas. Para não sobreinvestir tempo aqui: **3 a 4 frases por pergunta já valém nota cheia**, desde que sejam concretas. A pergunta 3 (métrica de impacto) é a que mais pesa na avaliação subjetiva — entreguem um número plausível com a unidade certa (hectares, toneladas, escolas, frotas), não um número exato pesquisado. **~20–30 min de discussão em grupo + ~15 min escrevendo** é o orçamento adequado de tempo para essa seção.

SITUAÇÃO	PONTUAÇÃO
4 perguntas respondidas com clareza, métricas concretas, articulação convincente entre tecnologia e impacto social	0,8–1,0
Pelo menos 3 perguntas respondidas, mas com alguma resposta vaga ou genérica	0,5–0,7
Resposta muito superficial — só uma ou duas perguntas tratadas, sem aprofundamento	0,2–0,4
Seção ausente, em branco, ou com texto que não responde nenhuma das perguntas	0,0–0,1

10.2 — Penalidades automáticas

SITUAÇÃO	PENALIZAÇÃO
Plágio ou cópia integral de outro grupo	Nota zero para todos os envolvidos
Projeto pronto reaproveitado sem adaptação ao contexto Mission Control	Nota zero
API Key versionada no histórico do Git (mesmo se removida depois)	–2,0 pts
Entrega com atraso e justificativa aprovada pelo professor (atestado médico ou similar)	–2,0 pts por dia de atraso
Entrega após 23h55 da data limite sem justificativa aprovada	Não aceita — equivale a não entregar
Vídeo acima de 3 minutos	–0,5 pt
README ausente ou em branco	–1,0 pt
Prints ausentes no README	–1,0 pt
Repositório privado ou inacessível na data de avaliação	–1,5 pts
Não usar Ollama Cloud sem justificativa documentada no README	–1,0 pt

Use este checklist na noite anterior à entrega — muitos pontos se perdem por esquecimento

Entrega

- ☐ Arquivo `.txt` com nomes completos e RMs de todos os integrantes
- ☐ Modalidade declarada (Individual / Dupla / Trio)
- ☐ Link do GitHub no arquivo `.txt` — testado e acessível
- ☐ Link do vídeo no arquivo `.txt` — testado em janela anônima
- ☐ Arquivo entregue no portal **dentro do prazo**

Repositório GitHub

- ☐ Repositório **público** e acessível sem login
- ☐ Código Python organizado (não tudo em um arquivo único)
- ☐ `requirements.txt` presente com versões fixadas
- ☐ `.env.example` presente, `.env` real **NÃO commitado**
- ☐ `.gitignore` configurado adequadamente
- ☐ README com todas as seções obrigatórias
- ☐ Pelo menos **2 prints reais** do sistema funcionando em `assets/`
- ☐ Commits regulares ao longo do desenvolvimento (não tudo no último commit)

Solução com IA

- ☐ Ollama Cloud (`gpt-oss:120b`) integrado e funcionando
- ☐ System prompt customizado com contexto da missão espacial

- ☐ Dados da telemetria são **injetados dinamicamente** no prompt (não hardcoded)
- ☐ Resposta da IA exibida de forma legível no output
- ☐ Persona definida e mantida coerente nas respostas

Funcionalidades da missão

- ☐ Monitoramento de pelo menos **3 parâmetros** (temperatura, energia, comunicação)
- ☐ Geração de **alertas automáticos** quando parâmetros estão fora do normal
- ☐ Lógica de tomada de decisão implementada em Python (não só no prompt)
- ☐ Resposta automatizada para pelo menos **uma situação crítica**
- ☐ Saída visualmente compreensível e organizada

Vídeo de demonstração

- ☐ Apresenta os integrantes do grupo no início (rosto, voz ou texto sobre frame)
- ☐ Demonstra o sistema rodando ao vivo (não slides)
- ☐ Mostra a IA gerando análise em tempo real
- ☐ Demonstra pelo menos um cenário de alerta ou crise
- ☐ Duração até 3 minutos
- ☐ Áudio audível e tela legível
- ☐ Configurado como **"Não listado"** no YouTube

Proposta de valor / modelo de negócio (NOVIDADE 2026.1)

- ☐ Seção " **Proposta de valor / modelo de negócio**" presente no [README.md](#)
- ☐ Articulado o **problema terrestre** que esta missão resolve no setor escolhido
- ☐ Definido **quem paga** pela solução (governo / privado / híbrido)
- ☐ **Métrica de impacto** concreta declarada (ex: hectares, CO₂, escolas conectadas)
- ☐ **Modelo de negócio** proposto (SaaS / dado-como-serviço / concessão / assinatura)

Datas e organização do tempo

Estimativa de esforço: ~5 horas de trabalho efetivo distribuídas

EVENTOS PRINCIPAIS

EVENTO	DATA SUGERIDA	STATUS
 Divulgação do desafio	A definir em aula	Confirmada em aula
 Formação dos grupos no portal	Até 1 semana após divulgação	Obrigatório
 Período de desenvolvimento	~3 semanas após divulgação	Trabalho contínuo
 Entrega final do .txt no portal	A definir em aula — 23h55	Sem prorrogação automática

As datas exatas serão confirmadas em aula conforme o calendário acadêmico de cada turma. **Não serão aceitas entregas fora do prazo** sem justificativa prévia aprovada pelo professor (atestado médico ou similar, conforme normas da FIAP).

12.1 — Sugestão de distribuição das 5 horas

HORA 1 — SETUP E PLANEJAMENTO

Criar conta Ollama, configurar .env, definir persona, escolher os parâmetros monitorados, esboçar fluxo do sistema.

HORA 2 — DADOS E LÓGICA BÁSICA

Implementar geração de telemetria, definir thresholds, escrever lógica de alertas em Python (sem IA ainda).

HORA 3 – INTEGRAÇÃO COM OLLAMA

Escrever system prompt, integrar a função `llm()`, testar com diferentes cenários, iterar no prompt.

HORA 4 – POLIMENTO E TESTES

Melhorar formatação da saída, testar todos os cenários, capturar prints, ajustar bugs.

HORA 5 – DOCUMENTAÇÃO E VÍDEO

Escrever README completo, organizar repositório, gravar vídeo de 3 minutos, submeter no portal.

BUFFER

Reserve mais 30-60 minutos como buffer para imprevistos — sempre acontecem.

13 • DICAS

Boas práticas e antipatterns

O que fazer e o que evitar — extraído de erros recorrentes em semestres anteriores

13.1 — Boas práticas (faça)

✓ COMECE PELO SYSTEM PROMPT

Antes de escrever 200 linhas de código, escreva o system prompt e teste algumas perguntas manualmente no ollama.com. Se o prompt não está bom isolado, ele não vai melhorar dentro do código.

✓ ITERE BASEADO EM TESTES REAIS

Sua primeira versão do prompt vai falhar em algum caso — isso é esperado. Documente no README as iterações ("v1 era muito vago, v2 ficou rígido demais, v3 ajustamos exemplos"). Esse processo vale pontos na avaliação.

✓ TESTE CENÁRIOS EXTREMOS

Não teste só o "tudo normal". Force temperatura impossível (500°C), energia negativa, módulo totalmente offline. Veja como o sistema responde e ajuste o prompt para esses casos.

✓ DOCUMENTE LIMITAÇÕES

Avaliadores valorizam grupos que sabem onde o próprio sistema falha. Inclua uma seção "Limitações conhecidas" no README. Honestidade técnica é diferencial profissional.

✓ VERSIONE CEDO, VERSIONE SEMPRE

Crie o repositório no **primeiro dia**, mesmo vazio. Cada integrante deve ter commits visíveis ao longo do projeto. Repositório que aparece com 1 commit gigante na noite da entrega levanta suspeita.

13.2 — Antipatterns (evite)

✗ SYSTEM PROMPT GENÉRICO

"Você é um assistente útil que analisa dados espaciais" não condiciona nada — o modelo continua respondendo qualquer coisa. Um bom system prompt define **papel + escopo + restrições + tom + formato de saída**.

✗ API KEY NO CÓDIGO VERSIONADO

Commitar a chave Ollama no GitHub é o erro mais comum e mais grave. A chave fica no histórico mesmo após remoção. Penalidade automática de -2 pontos, e risco real de uso não autorizado da sua conta.

✗ IA CHAMADA MAS DADOS NÃO USADOS

Grupos integram o LLM mas o prompt é sempre o mesmo (estático), sem injetar os dados reais da telemetria. Para o avaliador, isso parece "IA decorativa" e cai para 1,5–2,0 pts na Frente 3.

✗ LÓGICA TODA NO PROMPT

Delegar a decisão "é crítico ou não" para a IA sem nenhuma lógica Python é arriscado — o modelo pode ser inconsistente. Frente 4 espera **lógica em código**. A IA serve para **explicar e contextualizar**, não para substituir um `if`.

✗ ACEITAR A PRIMEIRA RESPOSTA DO LLM SEM TESTAR VARIAÇÕES

O modelo é **não-determinístico** — a mesma chamada com a mesma entrada pode retornar respostas diferentes. Grupos que rodam o sistema uma vez, gostam da resposta e seguem em frente costumam descobrir no vídeo (ou na correção) que o output varia bastante. Rodem **o mesmo cenário pelo menos 3 vezes**, observem a consistência, e ajustem temperatura, system prompt e exemplos até que a resposta fique estável dentro de uma faixa aceitável.

✗ VÍDEO LENDO ROTEIRO

Vídeos onde se nota claramente que o grupo está lendo texto perdem pontos. Prefiram falar com naturalidade, mesmo com pequenas pausas. Mostrem o sistema rodando ao vivo — isso vale mais que polimento de produção.

✗ TUDO NA NOITE ANTERIOR

Cinco horas de trabalho compactadas em uma única noite quase sempre resultam em entrega incompleta. Distribuem ao longo de pelo menos 3 sessões em dias diferentes. Histórico de commits no GitHub mostra isso.

14 • CONTATO

Suporte e dúvidas

Não fiquem travados — perguntem

Dúvidas sobre o desafio, ferramentas, ambiente, ou interpretação da rubrica devem ser direcionadas ao professor **antes da data de entrega** — depois não há como reverter situações. Os canais disponíveis são:

E-MAIL

Canal oficial para dúvidas individuais ou de equipe.
Resposta em até 48h em dias úteis.

profjorge.gomes@fiap.com.br

EM AULA

Dúvidas técnicas e de interpretação do enunciado são bem-vindas no início ou no final das aulas.
Aproveitem o tempo presencial.

SCRUM MASTER / COORDENAÇÃO

Para questões institucionais (formação de grupo no portal, atraso justificado, problemas técnicos do sistema FIAP).

PRÓPRIO CLAUDE / CHATGPT

Vocês podem (e devem) usar IA para tirar dúvidas técnicas durante o desenvolvimento — afinal, esse é o nosso objeto de estudo. Mas IA não pode **substituir** a entrega.

INTEGRIDADE ACADÊMICA – O QUE É PERMITIDO

Usar Claude, ChatGPT, Gemini para gerar rascunhos de prompts, código, README. 

Pedir a uma IA que revise seu system prompt. 

Discutir abordagens com colegas de outros grupos sem trocar código. 

Submeter o repositório de outro grupo como sendo de autoria do seu.  **Nota zero.**

Copiar projeto pronto da internet sem adaptação ao contexto Mission Control.  **Nota zero.**

"A IA não substitui o engenheiro. Ela amplia o alcance do julgamento técnico que vocês estão começando a construir agora — no terreno mais exigente que a engenharia já conheceu: o espaço."

FIAP · Ciência da Computação · Global Solution 2026.1
Disciplina: Prompt Engineering and Artificial Intelligence

Prof. Jorge Luiz Gomes · profjorge.gomes@fiap.com.br

#PromptAndArtificialIntelligence #FIAP #GlobalSolution #MissionControlAI #OllamaCloud

Copyright © 2026 Prof. Jorge Luiz Gomes — Todos os direitos reservados.
Reprodução total ou parcial deste documento sem consentimento formal é expressamente proibida.